Sintaxis strudell resumida

1. Introducción a Strudel

Strudel es un entorno de *live coding* musical que permite crear y modificar música escribiendo código. En lugar de producir música únicamente mediante una interfaz gráfica, se describen patrones mediante una sintaxis textual y funciones de JavaScript.

Su sintaxis permite expresar ritmos, notas, acordes, repeticiones, silencios, efectos y transformaciones temporales de forma compacta.

Para nuestro TFG, Strudel es una referencia para investigar cómo diseñar un lenguaje que permita escribir patrones musicales y transformarlos antes de enviar las instrucciones al motor de audio o a Reaper.

1.1. Conceptos básicos

En Strudel, un **patrón** describe eventos musicales distribuidos en el tiempo. Cada evento puede tener diferentes parámetros que determinan cómo se reproduce.

Concepto o parámetro	Significado
`note`	Permite especificar notas mediante nombres musicales, como `c4`, `e4` o `g4`.
`n`	Permite especificar valores numéricos que pueden interpretarse como grados de una escala.
`s`	Selecciona el sonido o *sample* que se reproduce.
`gain`	Controla el nivel de volumen.
`cutoff`	Controla la frecuencia de corte de un filtro.
`freq`	Permite especificar la frecuencia del sonido en hercios.
`scale()`	Permite interpretar grados numéricos dentro de una escala musical.

No todos estos elementos tienen la misma función: algunos generan patrones de eventos y otros modifican los parámetros de esos eventos.

```
note("c4 e4 g4").s("piano")
```

Define una secuencia de tres notas y utiliza el sonido de piano para reproducirlas.

```
n("0 2 4").scale("C:major")
```

Utiliza números que representan grados de la escala de Do mayor. La función `scale()` convierte esos grados en notas musicales.

Idea fundamental: un patrón describe qué eventos se producen y cuándo; los parámetros determinan cómo suenan.

---

2. Mini Notation

Documentación de Mini Notation

La *Mini Notation* es una sintaxis compacta diseñada para describir patrones rítmicos y musicales utilizando poco texto.

Permite expresar secuencias, silencios, repeticiones, subdivisiones, acordes y probabilidades sin tener que escribir una función de JavaScript para cada evento.

2.1. Elementos básicos de la sintaxis

Elemento	Función	Ejemplo	Interpretación
Espacio	Separa eventos consecutivos.	`note("c4 e4 g4")`	Tres eventos distribuidos dentro del ciclo.
`~`	Introduce un silencio.	`s("bd ~ bd ~")`	Alterna golpes de bombo y silencios.
`*n`	Repite o acelera un patrón dentro del mismo intervalo temporal.	`note("[e5 b4 d5 c5]*2")`	Reproduce la secuencia dos veces en el tiempo que ocuparía una vez.
`/n`	Alarga el patrón para que ocupe más ciclos.	`note("[e5 b4 d5 c5]/2")`	Distribuye la secuencia a lo largo de dos ciclos.
`< >`	Alterna entre diferentes elementos según el ciclo.	`note("<c4 e4 g4>")`	Selecciona un elemento distinto en cada ciclo.
`[ ]`	Agrupa eventos para tratarlos como una unidad.	`note("c4 [e4 g4] b4")`	Agrupa las notas interiores dentro del espacio asignado al grupo.
`,`	Combina eventos simultáneos.	`note("g3,b3,e4")`	Reproduce las tres notas como un acorde.
`@n`	Asigna un peso temporal a un evento o grupo.	`note("<[c4 e4]@2 [g4 b4]>")`	El primer grupo recibe un peso temporal mayor.
`_`	Permite alargar un evento mediante la notación de duración.	`note("c4 _ _ e4")`	Representa una nota sostenida a través de subdivisiones.
`!n`	Repite un evento un número determinado de veces.	`note("c4!2 e4")`	Repite el primer evento dos veces.
`?`	Introduce una condición de probabilidad.	`s("bd*8?")`	Cada evento tiene una probabilidad de aparecer.
`?p`	Especifica la probabilidad de aparición.	`s("bd*8?0.1")`	Utiliza una probabilidad de 0,1.

Elemento	Función	Ejemplo	Interpretación
`|`	Expresa alternativas entre patrones.	`note("c4 | e4 | g4")`	Permite seleccionar entre distintas alternativas.

Importante: los símbolos modifican cómo se distribuyen los eventos en el tiempo. No todos crean eventos nuevos: algunos agrupan, otros repiten y otros cambian su duración o probabilidad.

La duración real de un ciclo depende de la configuración temporal de Strudel. Un ciclo no equivale necesariamente a un segundo.

2.2. Agrupación y acordes

Los corchetes permiten agrupar eventos, mientras que las comas permiten reproducir notas simultáneamente.

```
note("<[g3,b3,e4] [a3,c4,e4] [b3,d4,f#4] [b3,e4,g4]>*2")
```

La expresión alterna entre cuatro acordes y repite la secuencia dos veces dentro del intervalo temporal correspondiente.

2.3. Ritmos euclídeos

Los ritmos euclídeos distribuyen un número determinado de pulsos de la manera más uniforme posible dentro de un número de posiciones.

```
s("bd(3,8)")
```

Parámetro	Significado
Beats	Número de pulsos que se van a distribuir.
Segments	Número total de posiciones disponibles.
Offset	Desplazamiento opcional de la distribución.

Ejemplo con un ritmo adicional:

```
s("bd(3,8,0), hh cp")
```

El bombo utiliza un ritmo euclídeo de tres pulsos en ocho posiciones. Los demás elementos se combinan con ese patrón.

Idea fundamental: Mini Notation permite describir estructuras musicales complejas mediante una sintaxis compacta. Es especialmente interesante para nuestro TFG porque puede facilitar la escritura de ritmos y secuencias sin necesidad de programar cada evento individualmente.

---

3. JavaScript API: construir patrones mediante funciones

Documentación de Pattern Factories

La JavaScript API permite construir patrones mediante funciones, en lugar de depender exclusivamente de la Mini Notation.
Esto facilita combinar patrones, reutilizar estructuras y crear abstracciones propias.

3.1. Equivalencia entre Mini Notation y funciones

Función	Equivalencia aproximada en Mini Notation	Descripción
`cat(x, y)`	`"<x y>"`	Concatena patrones, asignando un ciclo a cada elemento.
`seq(x, y)`	`"x y"`	Distribuye los elementos secuencialmente dentro de un ciclo.
`stack(x, y)`	`"x,y"`	Superpone patrones para que suenen simultáneamente.
`stepcat([3,x], [2,y])`	`"x@3 y@2"`	Concatena patrones asignando pesos temporales a cada elemento.
`polymeter([a,b,c], [x,y])`	`"{a b c, x y}"`	Combina patrones con estructuras métricas diferentes.
`silence`	`"~"`	Representa un patrón silencioso.
`run(n)`	Secuencia de `0` a `n-1`	Genera una secuencia de números consecutivos.
`binary(n)`	Patrón binario	Genera una estructura rítmica a partir de la representación binaria de un número.
`binaryN(n, bits)`	Patrón binario de longitud definida	Genera una representación binaria con un número de bits especificado.

Las equivalencias son orientativas: la interpretación exacta depende del contexto y de cómo se construye el patrón.

3.2. Generar una secuencia de notas

Con Mini Notation:

```
note("c3 eb3 g3")
```

Mediante funciones:

```
note(seq("c3", "eb3", "g3"))
```

Ambas expresiones permiten definir una secuencia de notas, pero la segunda hace explícita la construcción del patrón mediante `seq()`.

3.3. Combinar patrones

Un patrón puede construirse mediante varias funciones equivalentes.

Concatenar acordes:

```
cat(
  "g3,b3,e4",
  "a3,c4,e4",
  "b3,d4,f#4",
  "b3,e4,g4"
).note()
```

Construir cada acorde mediante `**stack()**`:

```
cat(
  stack("g3", "b3", "e4"),
  stack("a3", "c4", "e4"),
  stack("b3", "d4", "f#4"),
  stack("b3", "e4", "g4")
).note()
```

Utilizar Mini Notation:

```
note("<[g3,b3,e4] [a3,c4,e4] [b3,d4,f#4] [b3,e4,g4]>")
```

Las tres expresiones representan una progresión de cuatro acordes mediante mecanismos diferentes.

3.4. Mezclar ritmos individuales

Para combinar patrones rítmicos diferentes puede ser más cómodo utilizar funciones como `stack()`.

```
stack(
  note("c2 eb2(3,8)")
    .s("sawtooth")
    .cutoff(800),
  s("bd(5,8), hh*8")
)
```

En este ejemplo se superponen dos capas:

- Una secuencia de notas graves con sonido de diente de sierra y filtro.
- Una capa rítmica de bombo y hi-hat.

Cada capa conserva su propia estructura y puede tener sus propios parámetros.

Idea fundamental: las funciones permiten construir patrones de forma programática y combinarlos entre sí. Mini Notation ofrece una forma compacta de expresar estructuras similares, mientras que la API facilita crear operaciones y abstracciones reutilizables.

---

4. Value Modifiers: modificar los valores de los patrones

Documentación de Value Modifiers

Los *value modifiers* son funciones que transforman los valores de los eventos de un patrón.
En lugar de reescribir los datos, permiten aplicar operaciones matemáticas, redondear números, transformar rangos o interpretar proporciones.

Por ejemplo:

```
n("0 2 4".add(3)).scale("C:major")
```

Primero se suma tres a cada valor. Después, los nuevos valores se interpretan como grados de la escala de Do mayor.

4.1. Operaciones matemáticas

Función	Descripción	Ejemplo en Strudel	Resultado esperado
`add(valor)`	Suma una cantidad a cada valor.	`n("0 2 4".add(3)).scale("C:major")`	Los grados pasan a ser `3 5 7`.
`sub(valor)`	Resta una cantidad a cada valor.	`n("0 2 4".sub(1)).scale("C:major")`	Los grados pasan a ser `-1 1 3`.
`mul(valor)`	Multiplica cada valor por un factor.	`"<1 2 3>".mul(150).freq()`	Utiliza los valores multiplicados por 150 como frecuencias.
`div(valor)`	Divide cada valor por una cantidad.	`"<200 400 800>".div(2).freq()`	Utiliza las frecuencias `100`, `200` y `400` Hz.

4.2. Funciones de redondeo

Función	Descripción	Ejemplo en Strudel	Interpretación
`round()`	Redondea al entero más cercano.	`n("0.5 1.5 2.5".round()).scale("C:major")`	Redondea los valores antes de interpretarlos como grados.
`floor()`	Redondea hacia abajo.	`note("42 42.1 42.5 43".floor())`	Produce `42 42 42 43`.
`ceil()`	Redondea hacia arriba.	`note("42 42.1 42.5 43".ceil())`	Produce `42 43 43 43`.

4.3. Transformar rangos

Estas funciones son útiles para controlar parámetros cuyo valor cambia a lo largo del tiempo.

Función	Descripción	Ejemplo en Strudel	Aplicación
`range(min, max)`	Mapea una señal de 0 a 1 a un rango lineal.	`s("bd*4").cutoff(sine.range(500, 4000))`	Automatiza el filtro entre 500 y 4000 Hz.

Función	Descripción	Ejemplo en Strudel	Aplicación
`rangex(min, max)`	Mapea una señal de 0 a 1 mediante una curva exponencial.	`s("bd*4").cutoff(sine.rangex(500, 4000))`	Automatiza el filtro con una distribución exponencial.
`range2(min, max)`	Mapea una señal de -1 a 1 al rango indicado.	`s("bd*4").cutoff(sine2.range2(500, 4000))`	Permite utilizar una señal bipolar para controlar el filtro.

4.4. Otras funciones

Función	Descripción	Ejemplo en Strudel	Aplicación
`ratio()`	Permite trabajar con proporciones numéricas.	`ratio("1, 5:4, 3:2").mul(110).freq().s("piano")`	Genera frecuencias a partir de proporciones.
`as()`	Permite interpretar valores etiquetados como parámetros.	`"c:.5 a:1 f:.25 e:.8".as("note:clip")`	Representa varios valores asociados a parámetros.
Parámetros personalizados	Permiten definir nuevos controles.	Depende de la definición del parámetro.	Amplían las propiedades que se pueden controlar.

4.5. Encadenamiento de funciones

En Strudel es posible encadenar funciones para transformar un patrón y configurar su sonido.

```
note("a3 c#4 e4 a4")
  .s("sawtooth")
  .cutoff(500)
  .room(0.5)
```

El patrón se transforma progresivamente.

1. `note()` define las notas.
2. `s()` selecciona el sonido.
3. `cutoff()` modifica el filtro.
4. `room()` añade reverberación.

4.6. Crear funciones propias con `register()`

Strudel permite registrar funciones reutilizables.

```
const effectChain = register('effectChain', (pat) => pat
  .s("sawtooth")
  .cutoff(500)
  .room(0.5)
)

note("a3 c#4 e4 a4").effectChain()
```

En este ejemplo, `effectChain()` agrupa varias operaciones en una sola función. En vez de repetir la misma cadena de efectos, se define una vez y se reutiliza en otros patrones.

Idea fundamental: los *value modifiers* permiten transformar los valores de un patrón sin tener que reescribirlo. El encadenamiento y las funciones reutilizables permiten construir transformaciones más complejas a partir de operaciones sencillas.

---

5. Modificadores de tiempo (Time Modifiers)

Documentación de Time Modifiers

Los modificadores de tiempo permiten alterar la distribución temporal de un patrón sin tener que reescribir sus eventos.

Se pueden utilizar para acelerar, ralentizar, desplazar, invertir, repetir o seleccionar partes de una secuencia.

5.1. Velocidad y ritmo

Función	Equivalencia en Mini Notation	Descripción	Ejemplo
`slow(n)`	`/n`	Alarga el patrón para que ocupe más tiempo.	`"bd sd".slow(2)`
`fast(n)`	`*n`	Acelera el patrón y aumenta el número de repeticiones en el intervalo temporal.	`"bd sd".fast(2)`
`euclid(k,n)`	`(k,n)`	Distribuye `k` pulsos en `n` posiciones.	`"bd".euclid(3,8)`
`euclidRot(k,n,r)`	`(k,n,r)`	Aplica un ritmo euclídeo y desplaza su distribución.	`"bd".euclidRot(3,8,1)`
`euclidLegato(k,n)`	—	Genera un ritmo euclídeo en el que los eventos se mantienen hasta el siguiente.	`"bd".euclidLegato(3,8)`
`cpm(n)`	—	Establece la velocidad en ciclos por minuto.	`"bd sd".cpm(90)`

Los ritmos euclídeos distribuyen pulsos de manera uniforme; no se limitan a cambiar la velocidad de reproducción.

5.2. Desplazamiento temporal

Función	Descripción	Ejemplo
`early(n)`	Adelanta temporalmente los eventos.	`"bd sd".early(0.1)`
`late(n)`	Retrasa temporalmente los eventos.	`"bd sd".late(0.1)`
`swingBy(amount, n)`	Aplica swing a subdivisiones del patrón.	`"bd hh".swingBy(1/3, 4)`
`swing(n)`	Abreviatura de un swing con una cantidad de referencia de un tercio.	`"bd hh".swing(4)`

Estos modificadores permiten cambiar el *groove* sin cambiar necesariamente las notas o los sonidos.

5.3. Invertir y reorganizar patrones

Función	Descripción	Ejemplo
`rev()`	Invierte el orden temporal del patrón.	`"c4 e4 g4".rev()`
`palindrome()`	Alterna el patrón con su versión invertida.	`"c4 e4 g4".palindrome()`
`iter(n)`	Divide el patrón en subdivisiones y cambia progresivamente el punto de inicio.	`"bd sd hh cp".iter(4)`
`iterBack(n)`	Aplica una transformación similar a `iter()`, recorriendo las subdivisiones en sentido inverso.	`"bd sd hh cp".iterBack(4)`
`ply(n)`	Repite cada evento el número de veces indicado.	`"bd sd".ply(2)`
`fastGap(n)`	Acelera el patrón dejando un espacio vacío en el resto del ciclo.	`"bd sd".fastGap(2)`

5.4. Seleccionar y comprimir partes de un patrón

Función	Descripción	Ejemplo
`segment(n)`	Muestrea un patrón a un número de eventos por ciclo. Resulta útil para convertir señales continuas en eventos discretos.	`note(sine.range(40,52).segment(24))`
`compress(start,end)`	Comprime el patrón para que ocupe un intervalo temporal determinado.	`s("bd sd").compress(0.25,0.75)`
`zoom(start,end)`	Selecciona una parte del patrón y la reproduce	`"bd sd hh cp".zoom(0.25,0.75)`

Función	Descripción	Ejemplo
	dentro del intervalo temporal original.	
`linger(fraction)`	Selecciona una fracción del patrón y la prolonga para ocupar el intervalo correspondiente.	`"bd sd".linger(0.5)`
`clip(duration)`	Controla la duración de reproducción de los eventos o samples.	`"bd sd".clip(0.5)`
`ribbon(offset,duration)`	Permite repetir o mantener una porción del patrón mediante un desplazamiento temporal.	`s("bd!16?").ribbon(29,0.5)`

Los parámetros exactos y el comportamiento de algunas de estas funciones deben comprobarse con la documentación y los ejemplos oficiales antes de incorporarlos como especificaciones del proyecto.

5.5. Transformaciones dentro y fuera de un ciclo

Las funciones `inside()` y `outside()` permiten aplicar transformaciones temporales a distintas escalas.

```
"0 1 2 3 4 3 2 1"
  .inside(4, rev)
  .scale("C:major")
  .note()
```

Una transformación equivalente, expresada mediante modificadores de tiempo, es:

```
"0 1 2 3 4 3 2 1"
  .slow(4)
  .rev()
  .fast(4)
  .scale("C:major")
  .note()
```

La idea es que una operación puede aplicarse a una parte temporal del patrón o a una escala temporal más amplia.

Idea fundamental: los modificadores de tiempo permiten reutilizar una secuencia y cambiar su comportamiento temporal. Son útiles para crear variaciones rítmicas, modificar el *groove* y reorganizar patrones sin tener que escribirlos de nuevo.

---

6. Señales (Signals)

Documentación de Signals

Las **señales** (*signals*) son patrones que generan valores continuos, es decir, valores que pueden cambiar a lo largo del tiempo y que, teóricamente, tienen infinitos pasos.

A diferencia de una secuencia normal, que contiene eventos definidos en posiciones concretas, una señal genera un flujo de valores numéricos que puede utilizarse para modificar parámetros musicales.

Por ejemplo, podemos utilizar una señal para cambiar continuamente la frecuencia de corte de un filtro, el volumen o la altura de las notas.

6.1. Tipos de señales

Strudel incluye diferentes señales que generan valores siguiendo distintas formas de onda.

Señal	Descripción	Rango de valores	Ejemplo en Strudel
`saw`	Genera una onda de diente de sierra.	0 a 1	`n(saw.range(0,8).segment(8)).scale("C:major")`
`sine`	Genera una onda sinusoidal.	0 a 1	`n(sine.segment(16).range(0,15)).scale("C:minor")`
`cosine`	Genera una onda cosenoidal.	0 a 1	`n(cosine.segment(16).range(0,15)).scale("C:minor")`
`tri`	Genera una onda triangular.	0 a 1	`n(tri.segment(8).range(0,7)).scale("C:minor")`
`square`	Genera una onda cuadrada, alternando entre valores.	0 a 1	`n(square.segment(4).range(0,7)).scale("C:minor")`
`rand`	Genera valores aleatorios continuos.	0 a 1	`s("bd*4,hh*8").cutoff(rand.range(500,8000))`
`perlin`	Genera ruido Perlin, con variaciones aleatorias más suaves.	0 a 1	`s("bd*4,hh*8").cutoff(perlin.range(500,8000))`

Las señales permiten generar cambios automáticos sin tener que escribir manualmente todos los valores.

6.2. Señales con valores entre -1 y 1

También existen versiones de algunas señales que generan valores entre -1 y 1. Estas permiten trabajar con oscilaciones centradas en cero y utilizar transformaciones como `range2()`.

6.3. Convertir señales en eventos discretos

Una señal puede generar valores continuamente, pero en ocasiones necesitamos convertirla en una secuencia de eventos.

Para ello se puede utilizar `segment(n)`, que toma muestras de la señal y genera un número determinado de valores por ciclo.

```
n(sine.range(0, 8).segment(8))
  .scale("C:major")
```

En este ejemplo, la señal sinusoidal se transforma en ocho valores discretos y después se interpretan como grados de una escala.

6.4. Crear señales personalizadas

También se pueden definir señales mediante una función JavaScript:

```
signal(t => Math.pow(t,2) % 1)
  .range(40,60)
  .add(5)
  .segment(16)
  .note()
```

Esto permite definir comportamientos que no están incluidos en las señales predefinidas.

Idea fundamental: las señales permiten generar valores que evolucionan automáticamente con el tiempo. Son especialmente útiles para crear modulaciones y automatizar parámetros sin tener que definir manualmente cada cambio.

---

7. Random Modifiers

Documentación de Random Modifiers

Los *random modifiers* permiten introducir aleatoriedad en los patrones musicales.
Se pueden utilizar para seleccionar sonidos al azar, eliminar eventos de una secuencia o aplicar transformaciones con una probabilidad determinada.

7.1. Seleccionar elementos aleatoriamente

Función	Descripción	Ejemplo en Strudel	Aplicación
`choose()`	Selecciona aleatoriamente uno de los elementos de una lista.	`note("c2 g2!2 d2 f1").s(choose("sine","triangle","bd:6"))`	Alterna aleatoriamente entre diferentes sonidos.
`wchoose()`	Selecciona aleatoriamente un elemento asignando un	`note("c2 g2!2 d2 f1").s(wchoose(["sine",10],["triangle",1],["bd:6",1]))`	Permite que algunos sonidos tengan más

Función	Descripción	Ejemplo en Strudel	Aplicación
	peso a cada opción.		posibilidades de aparecer.

En `wchoose()`, cada elemento se acompaña de un peso. Un peso mayor significa que ese elemento tiene más posibilidades de ser seleccionado.

```
note("c2 g2 d2 f1")
  .s(wchoose(
    ["sine",10],
    ["triangle",1],
    ["bd:6",1]
  ))
```

En este caso, el sonido `sine` tiene un peso mucho mayor que los otros dos.

7.2. Seleccionar patrones aleatorios por ciclo

Estas funciones permiten elegir un patrón completo aleatoriamente en cada ciclo.

Función	Sinónimo	Descripción	Ejemplo en Strudel
`chooseCycles()`	`randcat()`	Selecciona aleatoriamente uno de los patrones en cada ciclo.	`chooseCycles("bd","hh","sd") .s().fast(8)`
`wchooseCycles()`	`wrandcat()`	Selecciona un patrón en cada ciclo utilizando pesos para controlar su probabilidad.	`wchooseCycles(["bd",10], ["hh",1],["sd",1]).s().fast(8)`

También se pueden utilizar directamente alternativas mediante Mini Notation:

```
s("bd | hh | sd").fast(8)
```

7.3. Eliminar eventos aleatoriamente

Los modificadores aleatorios también permiten eliminar eventos de un patrón o hacer que aparezcan solamente con cierta probabilidad.

Esto permite generar ritmos menos repetitivos sin tener que definir manualmente todas las variaciones.

7.4. Aplicar transformaciones con probabilidad

Función	Equivalencia	Probabilidad	Ejemplo
`often(fn)`	`sometimesBy(0.75, fn)`	75 %	`s("hh*8") .often(x=>x.speed("0.5"))`

Función	Equivalencia	Probabilidad	Ejemplo
`rarely(fn)`	`sometimesBy(0.25, fn)`	25 %	`s("hh*8").rarely(x=>x.speed("0.5"))`
`almostNever(fn)`	`sometimesBy(0.1, fn)`	10 %	`s("hh*8").almostNever(x=>x.speed("0.5"))`
`almostAlways(fn)`	`sometimesBy(0.9, fn)`	90 %	`s("hh*8").almostAlways(x=>x.speed("0.5"))`
`never(fn)`	`sometimesBy(0, fn)`	0 %	`s("hh*8").never(x=>x.speed("0.5"))`
`always(fn)`	`sometimesBy(1, fn)`	100 %	`s("hh*8").always(x=>x.speed("0.5"))`

Idea fundamental: los modificadores aleatorios permiten que un patrón no sea exactamente igual en cada reproducción. Son útiles para introducir variaciones, evitar repeticiones demasiado mecánicas y generar ritmos o melodías parcialmente impredecibles.

---

8. Conditional Modifiers

Documentación de Conditional Modifiers

Los *conditional modifiers* permiten aplicar transformaciones a un patrón solamente cuando se cumple una determinada condición.

Las condiciones pueden depender del número de ciclo, de una estructura binaria o de la posición de los eventos. También permiten dividir patrones, seleccionar partes de ellos, modificar acordes y controlar cuándo se reinician determinadas secuencias.

8.1. Aplicar transformaciones según el ciclo

Función	Descripción	Ejemplo en Strudel	Aplicación
`firstOf(n, fn)`	Aplica una transformación al principio de cada grupo de n ciclos.	`note("c3 d3 e3 g3").firstOf(4,x=>x.rev())`	Modifica el primer ciclo de cada grupo de cuatro.
`lastOf(n, fn)`	Aplica una transformación al último ciclo de cada grupo de n ciclos.	`note("c3 d3 e3 g3").lastOf(4,x=>x.rev())`	Modifica el último ciclo del grupo.
`when(pattern, fn)`	Aplica una transformación cuando el patrón indicado tiene un estado verdadero.	`"c3 eb3 g3".when("<0 1>/2",x=>x.sub("5")).note()`	Condiciona una transformación a otro patrón.

Ejemplo:

```
note("c3 d3 e3 g3")
  .firstOf(4, x => x.rev())
```

El patrón se invierte al principio de cada grupo de cuatro ciclos. En cambio, con `lastOf(4, ...)`, la inversión se aplica al último ciclo del grupo.

8.2. Dividir un patrón en partes

Estas funciones permiten dividir un patrón en varias secciones y aplicar transformaciones a cada una.

Función	Descripción	Aplicación
`chunk(n, fn)`	Divide el patrón en n partes y aplica una transformación a cada una.	Permite modificar secciones de un patrón de forma independiente.
`segment(n)`	Divide el patrón en un número determinado de eventos por ciclo.	Permite trabajar con subdivisiones concretas.

8.3. Aplicar estructuras binarias

Las estructuras binarias permiten definir condiciones a partir de secuencias de valores binarios. Esto resulta útil para crear máscaras rítmicas o controlar qué eventos se reproducen.

8.4. Modificar acordes y secuencias

Los modificadores condicionales también permiten aplicar transformaciones a los acordes o a partes concretas de una secuencia.

Esto facilita construir arpegios, modificar las notas de un acorde o cambiar una parte de la melodía sin tener que definir un patrón completamente nuevo.

8.5. Seleccionar patrones o funciones

Estas funciones permiten utilizar un patrón como selector para elegir qué elemento o transformación se aplica.

Función	Descripción	Ejemplo en Strudel
`pick(pattern, xs)`	Selecciona patrones o valores de una lista mediante índices o nombres. Mantiene la estructura del patrón original.	`note("<0 1 2!2 3>".pick(["g a","e f","f g f g","g c d"]))`
`pickmod(pattern, xs)`	Funciona como `pick()`, pero si el índice supera el tamaño de la lista, vuelve al principio.	Selección circular de elementos.
`pickF(pattern, functions)`	Selecciona qué función aplicar a otro patrón utilizando índices o nombres.	`s("bd [rim hh]").pickF("<0 1 2>", [rev,jux(rev),fast(2)])`
`pickmodF(pattern, functions)`	Funciona como `pickF()`, pero los índices superiores al	Selección circular de transformaciones.

Función	Descripción	Ejemplo en Strudel
	tamaño de la lista vuelven al principio.	
`pickRestart(pattern, xs)`	Selecciona un patrón y lo reinicia cuando se activa su índice.	Hace que cada selección comience desde el principio.
`pickmodRestart(pattern, xs)`	Versión circular de `pickRestart()`.	Combina selección circular y reinicio.
`pickReset(pattern, xs)`	Selecciona un patrón y lo resetea cuando se activa su índice.	Permite controlar el punto de reinicio.
`pickmodReset(pattern, xs)`	Versión circular de `pickReset()`.	Combina selección circular y reinicio.

Ejemplo de `pick()`:

```
note("<0 1 2!2 3>".pick([
  "g a",
  "e f",
  "f g f g",
  "g c d"
]))
```

Los números del patrón seleccionan los elementos correspondientes de la lista.

Ejemplo de `pickF()`:

```
s("bd [rim hh]")
  .pickF("<0 1 2>", [
    rev,
    jux(rev),
    fast(2)
  ])
```

El patrón selector determina qué transformación se aplica: invertir, aplicar `jux(rev)` o acelerar.

8.6. Seleccionar patrones comprimiendo su estructura

Función	Sinónimo	Descripción	Ejemplo
`inhabit(pattern, xs)`	`pickSqueeze()`	Selecciona patrones o valores de una lista o tabla de nombres y comprime los ciclos seleccionados para ajustarlos al patrón que los contiene.	`" <a b [a,b]> " .inhabit({a:s("bd(3,8 b:s("cp sd")})`
`inhabitmod(pattern, xs)`	`pickmodSqueeze()`	Funciona como `inhabit()`, pero	Selección circular.

Función	Sinónimo	Descripción	Ejemplo
		los índices superiores al tamaño de la lista vuelven al principio.	
`squeeze(pattern, xs)`	—	Selecciona un patrón o valor mediante índices y comprime el patrón seleccionado para ajustarlo a la duración del evento que lo selecciona.	`note (squeeze("<0@2 [1!2] 2>" ,["g a","f g f g","g a c d"]))`

Idea fundamental: los modificadores condicionales permiten controlar cuándo y cómo se transforma un patrón. Son útiles para crear variaciones estructuradas, arpegios, cambios por ciclo, máscaras rítmicas y secuencias que se reinician o seleccionan de forma controlada.

---

9. Accumulation Modifiers

Documentación de Accumulation Modifiers

Los *accumulation modifiers* permiten combinar un patrón original con versiones transformadas de ese mismo patrón.
En lugar de modificar únicamente la secuencia original, estas funciones pueden superponer otras versiones, crear capas, desplazar eventos en el tiempo o generar repeticiones similares a un eco.

Son especialmente útiles para crear armonías, duplicar melodías, añadir capas y generar efectos rítmicos.

9.1. Superponer y crear capas

Función	Descripción	Ejemplo en Strudel	Aplicación
`superimpose(fn)`	Superpone el resultado de aplicar una función al patrón original. Se reproducen tanto el patrón original como su versión transformada.	`"0 2 4 6" .superimpose(x=>x.add(2)) .scale("C:minor").note()`	Permite crear una segunda versión de una melodía, por ejemplo, transpuesta.
`layer(fn, ...)`	Crea capas con los resultados de las funciones indicadas. A diferencia de `superimpose()`, no conserva automáticamente el patrón original.	`"0 2 4 6" .layer(x=>x.add("0,2")) .scale("C:minor").note()`	Permite construir varias capas musicales a partir de un patrón.

Ejemplo de `superimpose()`:

```
"<0 2 4 6 ~ 4 ~ 2 0!3 ~!5>*8"
  .superimpose(x => x.add(2))
  .scale("C minor")
  .note()
```

El patrón original se reproduce junto con otra versión en la que se han sumado dos unidades a sus valores. Al aplicar la escala, ambas versiones se convierten en notas musicales.

Ejemplo de `layer()`:

```
"<0 2 4 6 ~ 4 ~ 2 0!3 ~!5>*8"
  .layer(x => x.add("0,2"))
  .scale("C minor")
  .note()
```

En este caso se construyen capas a partir del patrón, pero no se conserva automáticamente una capa original independiente como ocurre con `superimpose()`.

9.2. Desplazar patrones en el tiempo

Función	Descripción	Parámetros	Ejemplo en Strudel
`off(time, fn)`	Superpone el resultado de una transformación sobre el patrón original, pero desplazándolo temporalmente.	`time`: desplazamiento temporal. `fn`: función que se aplica al patrón.	`"c3 eb3 g3" .off(1/8,x=>x.add(7)).note()`

Ejemplo:

```
"c3 eb3 g3"
  .off(1/8, x => x.add(7))
  .note()
```

Se reproduce el patrón original y, además, una segunda versión transpuesta siete unidades y desplazada temporalmente una octava parte de ciclo.

9.3. Crear ecos y repeticiones

Función	Descripción	Parámetros	Ejemplo
`echo(times,time,feedback)`	Superpone varias repeticiones del patrón, desplazándolas en el tiempo y aplicando progresivamente	`times`: número de repeticiones. `time`: desplazamiento entre repeticiones.	`s("bd sd").echo(3,1/6,0.8)`

Función	Descripción	Parámetros	Ejemplo
	el parámetro de feedback.	`feedback`: multiplicador.	
`echoWith(times,time,fn)`	Crea varias superposiciones desplazadas en el tiempo y aplica una transformación diferente en cada repetición.	`times`: número de repeticiones. `time`: desplazamiento temporal. `fn`: función aplicada a cada repetición.	`"<0 [2 4]>" .echoWith(4,1/8, (p,n)=>p.add(n*2)) .scale("C:minor") .note()`
`stut(times,feedback,time)`	Versión antigua de `echo()`, con los dos últimos parámetros en un orden diferente.	`times`: número de repeticiones. `feedback`: multiplicador. `time`: desplazamiento temporal.	`s("bd sd") .stut(3,0.8,1/6)`

Ejemplo de `echo()`:

```
s("bd sd").echo(3,1/6,0.8)
```

Se generan tres repeticiones superpuestas, separadas por un sexto de ciclo. El parámetro `0.8` controla el multiplicador aplicado progresivamente a las repeticiones.

Ejemplo de `echoWith()`:

```
"<0 [2 4]>"
  .echoWith(4,1/8,(p,n)=>p.add(n*2))
  .scale("C:minor")
  .note()
```

En este caso, cada repetición recibe una transformación distinta. La función utiliza `n` como índice de la iteración y suma una cantidad diferente en cada repetición.

Importante: `stut()` está marcado como obsoleto (*deprecated*) en la documentación. Para código nuevo es preferible utilizar `echo()`.

Idea fundamental: los modificadores de acumulación permiten crear nuevas capas musicales a partir de un patrón existen